

""

StarFx Trading Signal Bot (fixed)

Required environment variables (set these before running – do NOT hardcode them):

TELEGRAM_TOKEN - your bot token from @BotFather

CHAT_ID - the chat/channel id to post signals to

Optional:

ACCOUNT_BALANCE - your real account balance in USD (default 10000.00 if unset)

STARFX_DB_PATH - path to the SQLite state file (default trading_data.db)

NOT FINANCIAL ADVICE: this is a signal-generation tool, not a validated trading system.

Several of the rules below (marked UNVALIDATED HEURISTIC) were not derived from a backtest and may not hold up going forward. Forward-test on a demo account before risking real capital, and treat position sizes as estimates to verify against your broker's actual contract specs.

Infra fixes:

1. Secrets loaded from environment variables instead of being hardcoded in

source.

2. State (active trades, daily stats, per-symbol cooldowns) persisted to SQLite so a

restart doesn't silently wipe open positions / stats.

3. Circuit breaker is re-checked immediately before sending an alert, closing the race

window where a setup could be evaluated after the daily loss limit was already hit.

4. All bare ``except: pass`` / ``except: print(...)`` replaced with logged exceptions.

Strategy fixes:

5. Data source: Kraken doesn't list XAU/USD or GBP/USD as spot pairs, so those calls

were silently failing. Everything now routes through Deriv (forex symbols use the `frXXXXYY` convention, e.g. `frxXAUUSD`, `frxGBPUSD`), with a startup check against

Deriv's `active_symbols` list that warns if a configured symbol isn't actually live.

6. "Engulfing" pattern now requires the candle's body to actually contain the prior candle's body, not just an opposite-colored close (the old check fired on any two-candle direction flip).

7. The weekend-R75-buy-only rule is kept but clearly flagged as an UNVALIDATED HEURISTIC (it was reverse-engineered

from a specific losing streak) and now logs every time it suppresses a signal, so you can see how often it actually fires.

8. Boom/Crash indices bake in periodic large spikes against their general drift (Boom

spikes up, Crash spikes down). The stop-loss is now widened specifically on the

side exposed to that spike direction (SELL on Boom, BUY on Crash).

9. ACCOUNT_BALANCE now reads from an env var instead of a hardcoded \$10,000 fiction.

10. Live spread (from Deriv's bid/ask tick) is folded into the risk distance used for

SL/TP math and position sizing, so displayed R:R is closer to what's achievable.

Commissions and swap are still not modeled – no data source for those here.

11. Position sizing display now reflects the real calculated risk-based size, with per-asset-class lot conventions (forex 100k units/lot, gold ~100oz/lot, synthetics

shown as raw units) instead of a hardcoded "0.001 Lots (Min)" placeholder.
"""

```
import os
os.environ['MPLCONFIGDIR'] = '/tmp'
os.environ['MPLBACKEND'] = 'Agg'
```

```
import asyncio
import json
import logging
import sqlite3
import sys
import threading
import time
from datetime import datetime, timezone
from http.server import HTTPServer,
BaseHTTPRequestHandler

import matplotlib
matplotlib.use('Agg')
import matplotlib.pyplot as plt
import mplfinance as mpf
import pandas as pd
from telegram import Update
from telegram.ext import Application,
CommandHandler, ContextTypes

# -----
# -----
# Logging
# -----
# -----
logging.basicConfig(
    level=logging.INFO,
    format="%(asctime)s [%(levelname)s] %(
(name)s: %(message)s",
)
log = logging.getLogger("starfx")

# -----
# -----
```

```

# Config / secrets
# -----
-----

TELEGRAM_TOKEN =
os.environ.get("TELEGRAM_TOKEN")
CHAT_ID = os.environ.get("CHAT_ID")

if not TELEGRAM_TOKEN or not CHAT_ID:
    log.critical(
        "TELEGRAM_TOKEN and CHAT_ID must be
set as environment variables. "
        "Refusing to start with
hardcoded/missing secrets."
    )
    sys.exit(1)

DB_PATH = os.environ.get("STARFX_DB_PATH",
"trading_data.db")

WEEKDAY_SYMBOLS = ["XAU/USD", "GBP/USD",
"R_75"]
WEEKEND_SYMBOLS = ["R_75", "R_100",
"BOOM1000", "CRASH1000"]
ACCOUNT_BALANCE =
float(os.environ.get("ACCOUNT_BALANCE",
"10000.00")) # fix #9
RISK_PER_TRADE_PCT = 0.01
MAX_DAILY_LOSS_PCT = 0.03
MAX_CONCURRENT_TRADES = 3

DERIV_WS_URL =
"wss://ws.derivws.com/websockets/v3?
app_id=1089"

```

```

# Deriv forex symbols use the frxXXXXYY convention; synthetics (R_75, BOOM1000, ...)
# already match Deriv's own codes and need no translation. Verified against Deriv's # active_symbols response format as of this writing (fix #5) – validate_symbols()
# double-checks this at startup rather than trusting it blindly.
DERIV_SYMBOL_MAP = {
    "XAU/USD": "frxXAUUSD",
    "GBP/USD": "frxGBPUSD",
}

def to_deriv_symbol(symbol):
    return DERIV_SYMBOL_MAP.get(symbol, symbol)

# UNVALIDATED HEURISTIC (fix #7): this rule was reverse-engineered from a specific # losing streak on R_75 over weekends, not derived from a backtest. It may or may not # hold going forward – every time it suppresses a signal it now logs so you can judge # whether it's actually earning its keep. Flip to False to disable it.
WEEKEND_R75_LONG_ONLY = True

# UNVALIDATED HEURISTIC (fix #8):
Boom/Crash indices bake in periodic large spikes

```

```

# against their general drift (Boom spikes
sharply up, Crash spikes sharply down). The
# side exposed to that spike gets a wider
stop to reduce (not eliminate) the chance
of
# being stopped out by a spike wick. The
1.6x multiplier is a starting point, not a
# backtested value – tune it against real
spike-size data for these instruments.
SPIKE_EXPOSED_SIDE = {"BOOM1000": "SELL",
"CRASH1000": "BUY"}
SPIKE_SL_BUFFER_MULT = 1.6

daily_stats = {"date": None,
"losses_today": 0.0, "is_circuit_broken":
False, "wins": 0, "losses": 0, "tp1_hits":
0}
active_trades = []
last_signal_time = {}

_state_lock = threading.Lock()

def is_paused():
    return os.path.exists("PAUSE") or
os.path.exists("pause")

def get_active_symbols():
    return WEEKEND_SYMBOLS if
datetime.now(timezone.utc).weekday() >= 5
else WEEKDAY_SYMBOLS

```

```

# -----
# Persistence (fix #2)
# -----

def init_db():
    conn = sqlite3.connect(DB_PATH)
    conn.execute("CREATE TABLE IF NOT
EXISTS trades (id INTEGER PRIMARY KEY
AUTOINCREMENT, symbol TEXT, bias TEXT,
result INTEGER)")
    conn.execute("CREATE TABLE IF NOT
EXISTS state (key TEXT PRIMARY KEY, value
TEXT)")
    conn.commit()
    conn.close()

def save_state():
    """Persist active_trades / daily_stats
/ last_signal_time so a restart doesn't
lose them."""
    try:
        serializable_trades = []
        for t in active_trades:
            t2 = {k: v for k, v in
t.items() if k != "df"} # DataFrame isn't
JSON-serializable
            serializable_trades.append(t2)

        stats_to_save = dict(daily_stats)
        if stats_to_save.get("date") is not
None:
            stats_to_save["date"] =

```

```

stats_to_save["date"].isoformat()

    with _state_lock:
        conn = sqlite3.connect(DB_PATH)
        conn.execute("INSERT OR REPLACE
INTO state (key, value) VALUES (?, ?)",
                    ("active_trades",
json.dumps(serializable_trades)))
        conn.execute("INSERT OR REPLACE
INTO state (key, value) VALUES (?, ?)",
                    ("daily_stats",
json.dumps(stats_to_save)))
        conn.execute("INSERT OR REPLACE
INTO state (key, value) VALUES (?, ?)",

("last_signal_time",
json.dumps(last_signal_time)))
        conn.commit()
        conn.close()
    except Exception:
        log.exception("Failed to save
state")

def load_state():
    global active_trades, daily_stats,
last_signal_time
    try:
        conn = sqlite3.connect(DB_PATH)
        rows = dict(conn.execute("SELECT
key, value FROM state").fetchall())
        conn.close()

        if "active_trades" in rows:

```

```

        active_trades =
json.loads(rows["active_trades"])
        log.info("Restored %d active
trade(s) from disk", len(active_trades))

        if "daily_stats" in rows:
            restored =
json.loads(rows["daily_stats"])
            if restored.get("date"):
                restored["date"] =
datetime.fromisoformat(restored["date"]).da
te()
                daily_stats.update(restored)
                log.info("Restored daily stats
from disk")

        if "last_signal_time" in rows:

last_signal_time.update(json.loads(rows["la
st_signal_time"]))
        except Exception:
            log.exception("Failed to load
state, starting fresh")

def record_trade_result(symbol, bias,
result):
    """result: 1 = win, 0 = loss"""
    try:
        conn = sqlite3.connect(DB_PATH)
        conn.execute("INSERT INTO trades
(symbol, bias, result) VALUES (?, ?, ?)",
(symbol, bias, result))
        conn.commit()

```

```

        conn.close()
    except Exception:
        log.exception("Failed to record
trade result")

# -----
# -----
# Health check server
# -----
# -----

class
HealthCheckHandler(BaseHTTPRequestHandler):
    def do_GET(self):
        self.send_response(200)
        self.end_headers()
        self.wfile.write(b"OK")

    def do_HEAD(self):
        self.send_response(200)
        self.end_headers()

    def log_message(self, format, *args):
        return

def start_dummy_server():
    HTTPServer(("0.0.0.0",
int(os.environ.get("PORT", 10000))),
HealthCheckHandler).serve_forever()

threading.Thread(target=start_dummy_server,
daemon=True).start()

```

```

# -----
# -----
# Formatting / sizing helpers
# -----
# -----

def format_price(s, p):
    if p is None:
        return "N/A"
    return f"{p:.2f}" if any(x in s for x
in ["XAU", "R_", "BOOM", "CRASH"]) else f"
{p:.5f}"

def calculate_position_size(balance,
risk_pct, risk_distance):
    """risk_distance is the price distance
being risked per unit – the caller decides
whether that already includes spread
(see fetch_spread /
evaluate_aplus_setup)."""
    return round((balance * risk_pct) /
risk_distance, 4) if risk_distance else 0

def format_lots(symbol, risk_distance,
balance=ACCOUNT_BALANCE,
risk_pct=RISK_PER_TRADE_PCT):
    """
    Fix #11: previously this returned a
hardcoded '0.001 Lots (Min) - Risk $20'
string
    for synthetic indices regardless of the

```

```

actual entry/SL distance. This now reports
    the real risk-based sizing, with per-
asset-class lot conventions – though the
exact
    contract size still varies by broker,
so treat this as an estimate to verify.
"""
    risk_amount = round(balance * risk_pct,
2)
    units =
calculate_position_size(balance, risk_pct,
risk_distance)
    if any(x in symbol for x in ["R_",
"BOOM", "CRASH"]):
        return f"{units:.4f} units (Risk
${risk_amount:.2f}) – verify against your
broker's min contract size"
    if "XAU" in symbol:
        lots = max(units / 100, 0.01) #
~100oz/lot on most brokers – confirm with
yours
        return f"{lots:.2f} Lots (Risk
${risk_amount:.2f}) – confirm your broker's
XAU contract size"
    lots = max(units / 100000, 0.01) #
100,000 units/lot for standard forex
    return f"{lots:.2f} Lots (Risk
${risk_amount:.2f})"

def validate_symbols():
    """Fix #5: query Deriv's active_symbols
list at startup and warn if any configured
symbol isn't actually live there,

```

```

instead of discovering it only when
fetch_data
    silently returns None during
scanning."""
    try:
        import websocket
        ws =
websocket.create_connection(DERIV_WS_URL,
timeout=10)

ws.send(json.dumps({"active_symbols":
"brief", "product_type": "basic"}))
        res = json.loads(ws.recv())
        ws.close()
        available = {s.get("symbol") for s
in res.get("active_symbols", [])}
        if not available:
            log.warning("active_symbols
lookup returned no symbols; skipping
validation: %s", res)
            return
        for sym in set(WEEKDAY_SYMBOLS) |
set(WEEKEND_SYMBOLS):
            d = to_deriv_symbol(sym)
            if d not in available:
                log.warning(
                    "Configured symbol '%s'
-> Deriv code '%s' was NOT found in Deriv's
"
                    "active_symbols list.
Signals for it will likely fail – verify
the "
                    "correct code in
Deriv's API docs.", sym, d,

```

```

    )
    except Exception:
        log.exception("Could not validate
symbols against Deriv active_symbols;
proceeding without validation")

def check_circuit_breaker():
    today =
datetime.now(timezone.utc).date()
    if daily_stats["date"] != today:
        daily_stats["date"] = today
        daily_stats["losses_today"] = 0.0
        daily_stats["is_circuit_broken"] =
False
        if daily_stats["losses_today"] >=
(ACCOUNT_BALANCE * MAX_DAILY_LOSS_PCT):
            daily_stats["is_circuit_broken"] =
True
        return False
    return True

# -----
# -----
# Data fetching
# -----
# -----
def fetch_data(symbol, tf, limit=100):
    """Fix #5: all symbols (forex +
synthetics) now come from Deriv. The
previous
version routed XAU/USD and GBP/USD
through ccxt's Kraken client, but Kraken is

```

```

a
    crypto exchange and doesn't list either
    as a spot pair – that call was silently
    failing every time, so weekday forex
    signals were effectively dead.""
    try:
        import websocket
        gran_map = {'1m': 60, '5m': 300,
                    '15m': 900, '1h': 3600, '4h': 14400, '1d':
                    86400}
        deriv_symbol =
        to_deriv_symbol(symbol)
        ws =
        websocket.create_connection(DERIV_WS_URL,
        timeout=10)
        ws.send(json.dumps({
            "ticks_history": deriv_symbol,
            "adjust_start_time": 1,
            "count": limit,
            "end": "latest",
            "granularity": gran_map.get(tf,
300),
            "style": "candles",
        })))
        res = json.loads(ws.recv())
        ws.close()
        if "candles" not in res:
            log.warning("No candles for %s
(Deriv code %s): %s", symbol, deriv_symbol,
res.get("error", res))
            return None
        df = pd.DataFrame(res["candles"])
        df['timestamp'] =
        pd.to_datetime(df['epoch'], unit='s',

```

```

utc=True)
    df.set_index('timestamp',
inplace=True)
    df = df[['open', 'high', 'low',
'close']].astype(float)
    df['volume'] = 1000
    return df
except Exception:
    log.exception("fetch_data failed
for %s", symbol)
    return None

def fetch_tick_full(symbol):
    """Single-snapshot tick (subscribe:0)
with bid/ask when Deriv provides them, used
both for live price and for spread
estimation (fix #10)."""
    try:
        import websocket
        d = to_deriv_symbol(symbol)
        ws =
websocket.create_connection(DERIV_WS_URL,
timeout=5)
        ws.send(json.dumps({"ticks": d,
"subscribe": 0}))
        res = json.loads(ws.recv())
        ws.close()
        tick = res.get("tick")
        if not tick:
            return None
        quote = tick.get("quote")
        ask = tick.get("ask")
        bid = tick.get("bid")

```

```

        return {
            "quote": float(quote) if quote
is not None else (float(ask) if ask is not
None else None),
            "ask": float(ask) if ask is not
None else None,
            "bid": float(bid) if bid is not
None else None,
        }
    except Exception:
        log.exception("fetch_tick_full
failed for %s", symbol)
        return None

def fetch_current_price(symbol):
    tick = fetch_tick_full(symbol)
    if tick and tick.get("quote") is not
None:
        return tick["quote"]
    df = fetch_data(symbol, '1m', 2)
    return df['close'].iloc[-1] if df is
not None else None

def fetch_spread(symbol):
    """Fix #10: rough round-trip cost
estimate from live bid/ask. Returns 0.0
(not None)
    when Deriv doesn't expose bid/ask for a
symbol (common for synthetic indices, which
often quote a single price) – callers
treat that as "no known spread", not an
error.

```

```

        This does not account for commissions
        or swap; there's no data source for those
        here."""
        tick = fetch_tick_full(symbol)
        if tick and tick.get("ask") is not None
        and tick.get("bid") is not None:
            return max(tick["ask"] -
            tick["bid"], 0.0)
        return 0.0

def generate_tradingview_chart(df, symbol,
setup, filename=None):
    if filename is None:
        filename =
f"chart_{symbol.replace('/',
'_')}_int(time.time())}.png"
        plot_df = df.iloc[-60:].copy()
        mc =
mpf.make_marketcolors(up='#26a69a',
down='#ef5350', edge='inherit',
wick='inherit', volume='in')
        style =
mpf.make_mpf_style(marketcolors=mc,
gridstyle=":", gridcolor="#2a2e39",
facecolor="#131722")
        disp = symbol.replace("R_75",
"Volatility 75 Index").replace("R_100",
"Volatility 100 Index")
        fig, _ = mpf.plot(
            plot_df, type='candle',
            style=style,
            title=f"\n{disp} -
{setup['bias']}",

```

```

        hlines=dict(
            hlines=[setup['price'],
                    setup['tp1'], setup['tp2'], setup['sl']],
            colors=['#2962ff', '#00e676',
                   '#00c853', '#ff1744'],
            linestyle='--', linewidths=1.2,
        ),
        savefig=filename, returnfig=True,
        figratio=(16, 9), figscale=1.2,
    )
    plt.close(fig)
    return filename

# -----
# -----
# Strategy
# -----
# -----

def analyze_structure(df, window=20):
    if df is None or len(df) < window + 1:
        return "NEUTRAL"
    recent = df.iloc[: -1]
    high = recent['high'].iloc[-
window:].max()
    low = recent['low'].iloc[-
window:].min()
    curr = df['close'].iloc[-1]
    if curr > high:
        return "BULLISH"
    if curr < low:
        return "BEARISH"
    ema =
df['close'].ewm(span=20).mean().iloc[-1]

```

```

        return "BULLISH" if curr > ema else
        "BEARISH"

def detect_price_action(df):
    if df is None or len(df) < 3:
        return None
    c1 = df.iloc[-2] # most recently
    closed candle
    c0 = df.iloc[-3] # candle before that
    body = abs(c1['close'] - c1['open'])
    uw = c1['high'] - max(c1['close'],
    c1['open'])
    lw = min(c1['close'], c1['open']) -
    c1['low']

    if lw >= (2 * body) and uw <= (0.5 *
    body):
        return "BULLISH_PINBAR"
    if uw >= (2 * body) and lw <= (0.5 *
    body):
        return "BEARISH_PINBAR"

    # Fix #6: a true engulfing candle must
    fully contain the prior candle's body, not
    # just close in the opposite direction
    – the old check fired on any two-candle
    # direction flip, which is a much
    weaker and noisier signal than real
    engulfing.
    prev_bearish = c0['close'] < c0['open']
    prev_bullish = c0['close'] > c0['open']
    curr_bullish = c1['close'] > c1['open']
    curr_bearish = c1['close'] < c1['open']

```

```

        if curr_bullish and prev_bearish and
c1['open'] <= c0['close'] and c1['close']
>= c0['open']:
            return "BULLISH_ENGULFING"
        if curr_bearish and prev_bullish and
c1['open'] >= c0['close'] and c1['close']
<= c0['open']:
            return "BEARISH_ENGULFING"
        return None

def evaluate_a_plus_setup(symbol):
    if not check_circuit_breaker() or
len(active_trades) >=
MAX_CONCURRENT_TRADES:
        return None
    h4 = fetch_data(symbol, '4h')
    h1 = fetch_data(symbol, '1h')
    m5 = fetch_data(symbol, '5m')
    if h4 is None or h1 is None or m5 is
None:
        return None
    h4_bias = analyze_structure(h4)
    h1_bias = analyze_structure(h1)
    if h4_bias != h1_bias or h4_bias ==
"NEUTRAL":
        return None
    m5_pa = detect_price_action(m5)
    if not m5_pa:
        return None

    bias = "BUY" if "BULLISH" in m5_pa else
"SELL"

```

```

        # Fix #7: UNVALIDATED HEURISTIC, kept
        from the original but now logged every time
        it
        # actually suppresses something, so you
        can judge whether it's earning its keep.
        if "R_75" in symbol and
        datetime.now(timezone.utc).weekday() >= 5
        and WEEKEND_R75_LONG_ONLY and bias ==
        "SELL":
            log.info("Suppressed %s SELL
            signal: weekend-R75-long-only filter
            (unvalidated heuristic)", symbol)
            return None

        price = m5['close'].iloc[-1]
        atr = (m5['high'] -
        m5['low']).rolling(14).mean().iloc[-1]
        if pd.isna(atr) or atr == 0:
            atr = price * 0.001

        if "R_75" in symbol:
            min_sl = price * 0.003
            mult = 5.0
        elif any(x in symbol for x in ["R_",
        "BOOM", "CRASH"]):
            min_sl = price * 0.002
            mult = 3.0
        elif "XAU" in symbol:
            min_sl = 2.5
            mult = 1.8
        else:
            min_sl = atr * 1.2
            mult = 1.2

```

```

        if "R_75" in symbol or "R_" in symbol
or "BOOM" in symbol or "CRASH" in symbol or
"XAU" in symbol:
            sl = (m5['low'].iloc[-3:].min() -
max(min_sl, atr * mult)) if bias == "BUY" \
            else
(m5['high'].iloc[-3:].max() + max(min_sl,
atr * mult))
        else:
            sl = (m5['low'].iloc[-3:].min() -
atr * mult) if bias == "BUY" \
            else
(m5['high'].iloc[-3:].max() + atr * mult)

        # Fix #8: widen the stop on the side of
Boom/Crash indices exposed to their built-
in
        # spike direction. Reduces, does not
eliminate, the chance of a spike wick
stopping
        # the trade out before the underlying
trend plays out. Tune this against real
        # spike-size data – the 1.6x starting
multiplier is not backtested.
        for key, exposed_bias in
SPIKE_EXPOSED_SIDE.items():
            if key in symbol and bias ==
exposed_bias:
                base_risk = abs(price - sl)
                extra = base_risk *
(SPIKE_SL_BUFFER_MULT - 1)
                sl = (sl - extra) if bias ==
"BUY" else (sl + extra)

```

```

        log.info("Applied spike-risk SL
buffer to %s %s (widened %.1fx)", symbol,
bias, SPIKE_SL_BUFFER_MULT)

    raw_risk = abs(price - sl)
    if raw_risk <= 0:
        return None

    # Fix #10: fold the live spread into
the risk distance used for SL/TP math and
    # position sizing, so displayed R:R is
closer to what's actually achievable.
    # fetch_spread returns 0.0 (not None)
when Deriv doesn't expose bid/ask for a
symbol.
    spread = fetch_spread(symbol)
    effective_risk = raw_risk + spread
    if effective_risk <= 0:
        return None

    if bias == "BUY":
        tp1, tp2 = price + effective_risk *
1.5, price + effective_risk * 3.0
    else:
        tp1, tp2 = price - effective_risk *
1.5, price - effective_risk * 3.0

    return {
        "symbol": symbol, "bias": bias,
"price": price, "sl": sl,
        "tp1": tp1, "tp2": tp2, "spread":
spread, "risk_distance": effective_risk,
        "position_units":
calculate_position_size(ACCOUNT_BALANCE,

```

```

RISK_PER_TRADE_PCT, effective_risk),
    "df": m5, "tp1_hit": False,
    }

# -----
# -----
# Background tasks
# -----
# -----

async def track_positions(app):
    while True:
        try:
            if is_paused():
                await asyncio.sleep(60)
                continue
            for trade in
list(active_trades):
                cur =
fetch_current_price(trade['symbol'])
                if cur is None:
                    continue
                sym = trade['symbol']
                disp = sym.replace("R_75",
"Volatility 75 Index").replace("R_100",
"Volatility 100 Index")
                ep = format_price(sym,
trade['price'])
                slp = format_price(sym,
trade['sl'])
                tp1p = format_price(sym,
trade['tp1'])
                tp2p = format_price(sym,
trade['tp2'])

```

```

curr_p = format_price(sym,
cur)

hit_tp1 = (trade['bias'] ==
"BUY" and cur >= trade['tp1']) or
(trade['bias'] == "SELL" and cur <=
trade['tp1'])
hit_tp2 = (trade['bias'] ==
"BUY" and cur >= trade['tp2']) or
(trade['bias'] == "SELL" and cur <=
trade['tp2'])
hit_sl = (trade['bias'] ==
"BUY" and cur <= trade['sl']) or
(trade['bias'] == "SELL" and cur >=
trade['sl'])

if not trade['tp1_hit'] and
hit_tp1 and not hit_tp2:
trade['tp1_hit'] = True
daily_stats['tp1_hits']
+= 1

try:
await
app.bot.send_message(chat_id=CHAT_ID,
text=f"✅ TP1 HIT (1:1.5) {disp}\nEntry
{ep} -> Now {curr_p}\nTP1 {tp1p} Secured")
except Exception:

log.exception("Failed to send TP1 message
for %s", sym)

save_state()
elif hit_tp2:
daily_stats['wins'] +=
1

```

```

record_trade_result(sym, trade['bias'], 1)
    try:
        await
    app.bot.send_message(chat_id=CHAT_ID,
text=f"🎯🎯 TP2 HIT FULL (1:3)
{disp}\nEntry {ep} -> TP2 {tp2p}")
    except Exception:

log.exception("Failed to send TP2 message
for %s", sym)

active_trades.remove(trade)
    save_state()
    elif hit_sl:
        if not
trade['tp1_hit']:

daily_stats['losses'] += 1

daily_stats['losses_today'] +=
ACCOUNT_BALANCE * RISK_PER_TRADE_PCT

record_trade_result(sym, trade['bias'], 0)
    try:
        await
    app.bot.send_message(chat_id=CHAT_ID,
text=f"🔴 SL HIT {disp}\nEntry {ep} SL
{slp} Now {curr_p}")
    except Exception:

log.exception("Failed to send SL message
for %s", sym)

```

```

active_trades.remove(trade)
            save_state()
            await asyncio.sleep(10)
        except Exception:
            log.exception("track_positions
loop error")
            await asyncio.sleep(10)

async def market_scanner(app):
    last_m5 = {}
    while True:
        try:
            if is_paused():
                await asyncio.sleep(60)
                continue
            for symbol in
get_active_symbols():
                if symbol in
last_signal_time and time.time() -
last_signal_time[symbol] < 3600:
                    continue
                if any(t['symbol'] ==
symbol for t in active_trades):
                    continue
                m5_df = fetch_data(symbol,
'5m', 5)
                if m5_df is None:
                    continue
                cur = m5_df.index[-1]
                if last_m5.get(symbol) ==
cur:
                    continue
                last_m5[symbol] = cur

```

```

        setup =
evaluate_aplus_setup(symbol)
        if not setup:
            continue
        if any(t['symbol'] ==
symbol and t['bias'] == setup['bias'] for t
in active_trades):
            continue

        # Fix #3: re-check the
circuit breaker right before sending,
closing the
        # race window between
evaluation and dispatch.
        if not
check_circuit_breaker():
            log.warning("Circuit
breaker tripped just before sending signal
for %s; skipping.", symbol)
            continue

        chart =
generate_tradingview_chart(setup['df'],
symbol, setup)
        ep = format_price(symbol,
setup['price'])
        slp = format_price(symbol,
setup['sl'])
        tp1p = format_price(symbol,
setup['tp1'])
        tp2p = format_price(symbol,
setup['tp2'])
        disp =

```

```

(symbol.replace("R_75", "Volatility 75
Index")

.replace("R_100", "Volatility 100 Index")

.replace("BOOM1000", "Boom 1000")

.replace("CRASH1000", "Crash 1000"))
        cap = (f"🎯 {setup['bias']}}
{disp}}\nEntry {ep}}\nSL {slp}}\nTP1 {tp1p}
(1:1.5)\nTP2 {tp2p} (1:3)\n"
                f"Size
{format_lots(symbol,
setup['risk_distance'])}}\nMode: WEEKEND")
        try:
            with open(chart, "rb")
as photo:
                await
app.bot.send_photo(chat_id=CHAT_ID,
photo=photo, caption=cap)

active_trades.append({k: v for k, v in
setup.items() if k != "df"})

last_signal_time[symbol] = time.time()
        save_state()
        except Exception:
            log.exception("Failed
to send signal for %s", symbol)
        finally:
            if
os.path.exists(chart):
                os.remove(chart)
            await asyncio.sleep(3)

```

```

        await asyncio.sleep(30)
    except Exception:
        log.exception("market_scanner
loop error")
        await asyncio.sleep(10)

async def schedule_daily_report(app):
    sent = None
    while True:
        try:
            now =
datetime.now(timezone.utc)
            if now.hour == 20 and
now.minute <= 10 and sent != now.date():
                total = daily_stats['wins']
+ daily_stats['losses'] +
daily_stats['tp1_hits']
                rw = daily_stats['wins'] +
daily_stats['tp1_hits']
                wr = rw / total * 100 if
total > 0 else 0
                rep = (f"📊 DAILY REPORT
23:00 EAT {now.date()}\nMode WEEKEND 1H no
dup BUY-only V75\n"
                        f"WR {wr:.1f}% Wins
{rw} Losses {daily_stats['losses']} TP1
{daily_stats['tp1_hits']} "
                        f"TP2
{daily_stats['wins']}\nActive
{len(active_trades)}")
                try:
                    await
app.bot.send_message(chat_id=CHAT_ID,

```

```

text=rep)

        sent = now.date()
    except Exception:
        log.exception("Failed
to send daily report")
        await asyncio.sleep(60)
    except Exception:

log.exception("schedule_daily_report loop
error")

        await asyncio.sleep(60)

# -----
-----
# Telegram commands
# -----
-----
async def start_command(update: Update,
context: ContextTypes.DEFAULT_TYPE):
    status = "🛑 PAUSED" if is_paused()
else "▶️ RUNNING"
    await update.message.reply_text(
        f"StarFx V11 {status} BUY-only V75
weekend\nCooldown 1H no dup\n"
        f"Active: {'',
'.join(get_active_symbols())}\n/signal
/price /report"
    )

async def signal_command(update: Update,
context: ContextTypes.DEFAULT_TYPE):
    await

```

```

update.message.reply_text(f"Scanning {'',
'.join(get_active_symbols())}...")
    found = 0
    for sym in get_active_symbols():
        setup = evaluate_a_plus_setup(sym)
        if setup:
            found += 1
            chart =
generate_tradingview_chart(setup['df'],
sym, setup)
            ep = format_price(sym,
setup['price'])
            try:
                with open(chart, "rb") as
photo:
                    await
context.bot.send_photo(chat_id=update.effective_chat.id, photo=photo, caption=f"
{setup['bias']} {sym} {ep}")
            except Exception:
                log.exception("Failed to
send manual signal for %s", sym)
            finally:
                if os.path.exists(chart):
                    os.remove(chart)
    if found == 0:
        await update.message.reply_text("No
A+ setup now.")

async def price_command(update: Update,
context: ContextTypes.DEFAULT_TYPE):
    msg = "💰 Live Prices:\n"
    for sym in get_active_symbols():

```

```

        p = fetch_current_price(sym)
        if p:
            msg += f"{sym}:"
{format_price(sym, p)}\n"
        await update.message.reply_text(msg)

async def report_command(update: Update,
context: ContextTypes.DEFAULT_TYPE):
    total = daily_stats['wins'] +
daily_stats['losses'] +
daily_stats['tp1_hits']
    rw = daily_stats['wins'] +
daily_stats['tp1_hits']
    wr = rw / total * 100 if total > 0 else
0
    rep = (f"📊 MANUAL REPORT
{datetime.now(timezone.utc).date()} WEEKEND
V11 BUY-only V75\n"
          f"WR {wr:.1f}% Wins {rw} Losses
{daily_stats['losses']} TP1
{daily_stats['tp1_hits']} "
          f"TP2
{daily_stats['wins']}\nActive
{len(active_trades)} Status {'PAUSED' if
is_paused() else 'RUNNING'}")
    await update.message.reply_text(rep)

# -----
# -----
# Entrypoint
# -----
# -----

```

```
async def main():
    init_db()
    load_state()
    validate_symbols()

    app =
Application.builder().token(TELEGRAM_TOKEN)
.build()
    app.add_handler(CommandHandler("start",
start_command))

app.add_handler(CommandHandler("signal",
signal_command))
    app.add_handler(CommandHandler("scan",
signal_command))
    app.add_handler(CommandHandler("price",
price_command))

app.add_handler(CommandHandler("report",
report_command))
    app.add_handler(CommandHandler("daily",
report_command))

asyncio.create_task(market_scanner(app))

asyncio.create_task(track_positions(app))

asyncio.create_task(schedule_daily_report(a
pp))

    log.info("StarFx bot online")
    async with app:
        await app.initialize()
```

```
    await app.start()
    await app.updater.start_polling()
    await asyncio.Event().wait()
```

```
if __name__ == "__main__":
    asyncio.run(main())
```